



Operating Systems

[1500WETOPS]

José Oramas

PSA

Exercise Sessions: Tomorrow (04/11)

- Complementary Session on Paging



Processor Scheduling

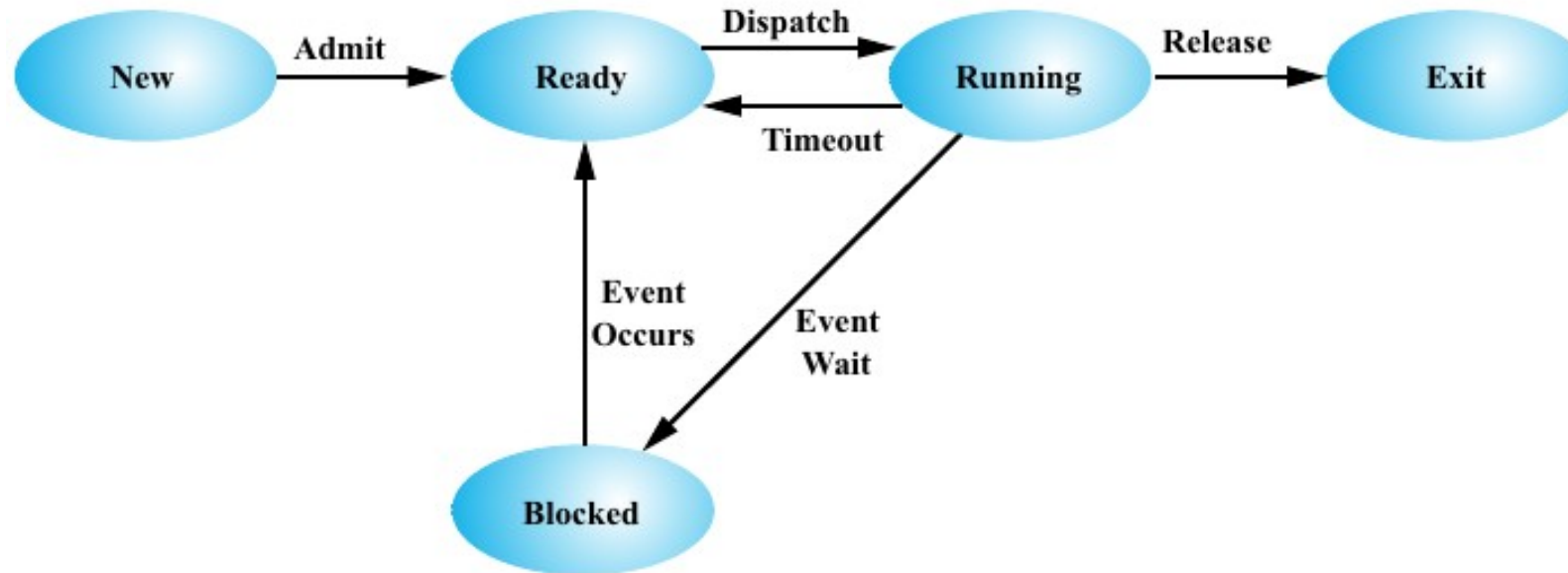
[Deciding who can use the processor]

José Oramas

Processor Scheduling

Introduction

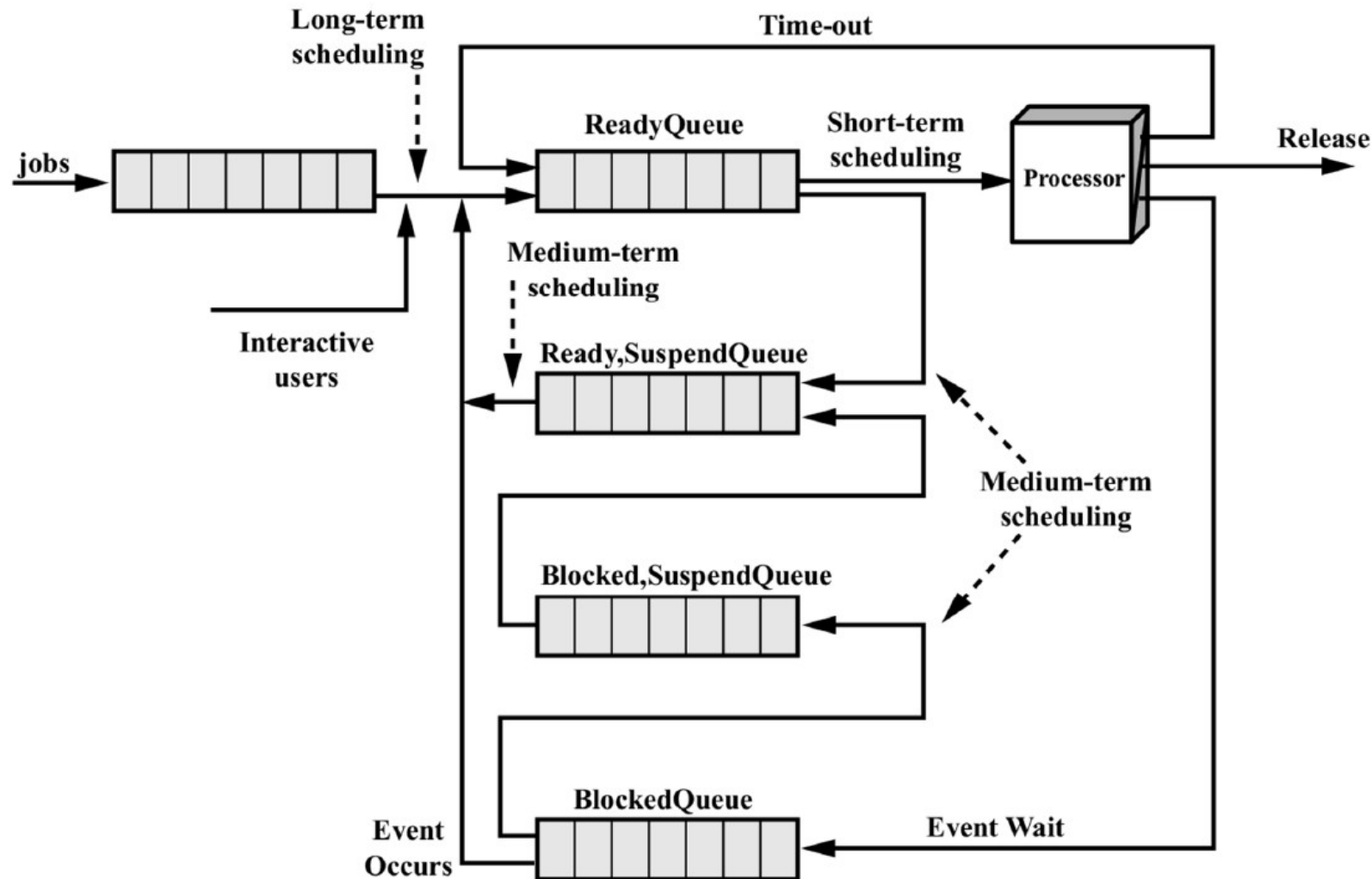
- Scheduling mechanisms will determine **how the state of a process evolves**.
- Three sub-problems: *long*, *medium* and *short* -term scheduling.



Processor Scheduling

Long-Term Scheduling

- New → Ready state.
- Decisions:
 - When to activate a process?
 - Which process to activate?
- Monitors idle state of the processor.
- Processor vs. I/O-bound processes



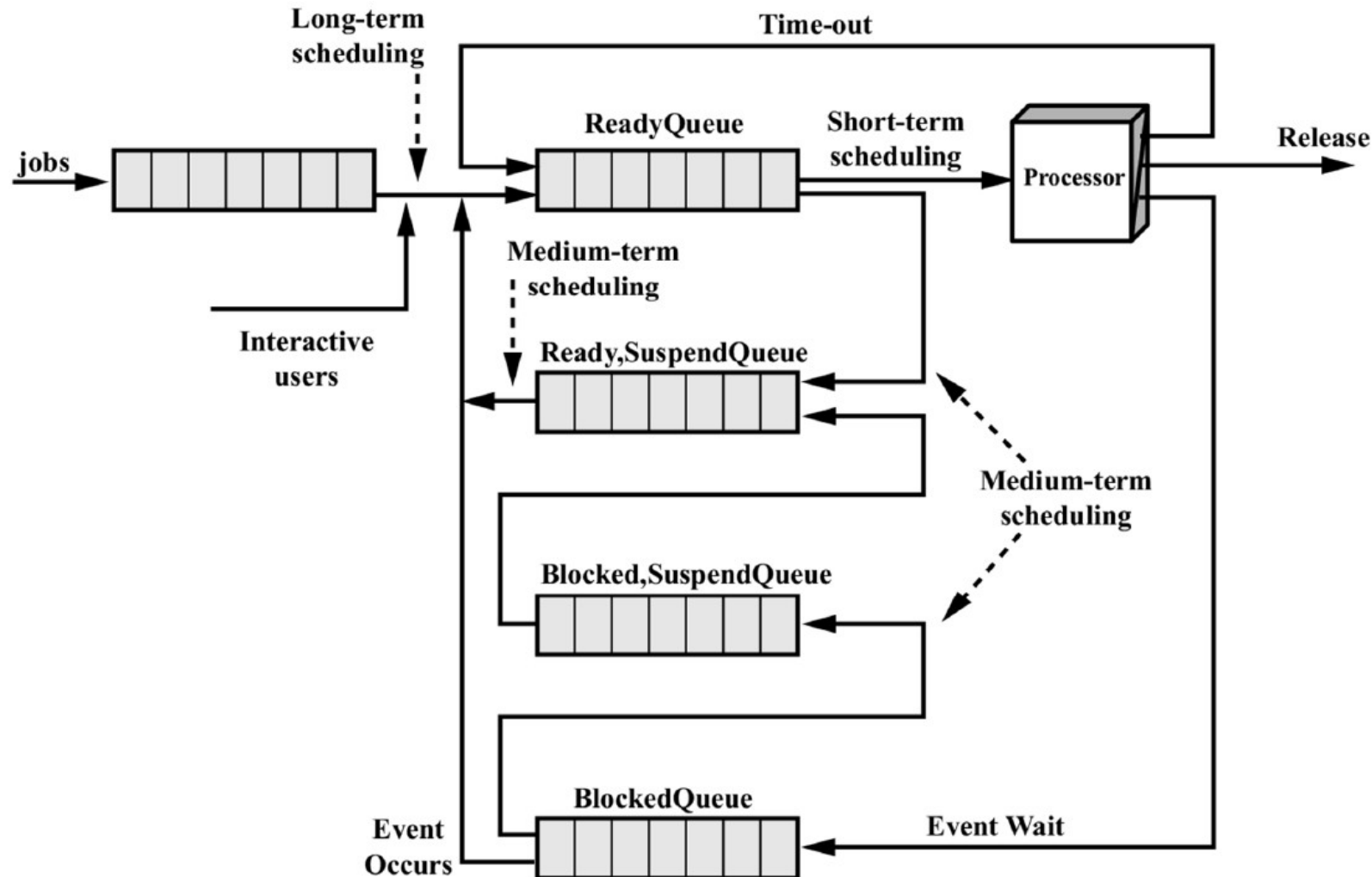
Processor Scheduling

What would happen if ...

- Too many processes (in general) are active ?
- I have too many processor or I/O bound processes ?

Long-Term Scheduling

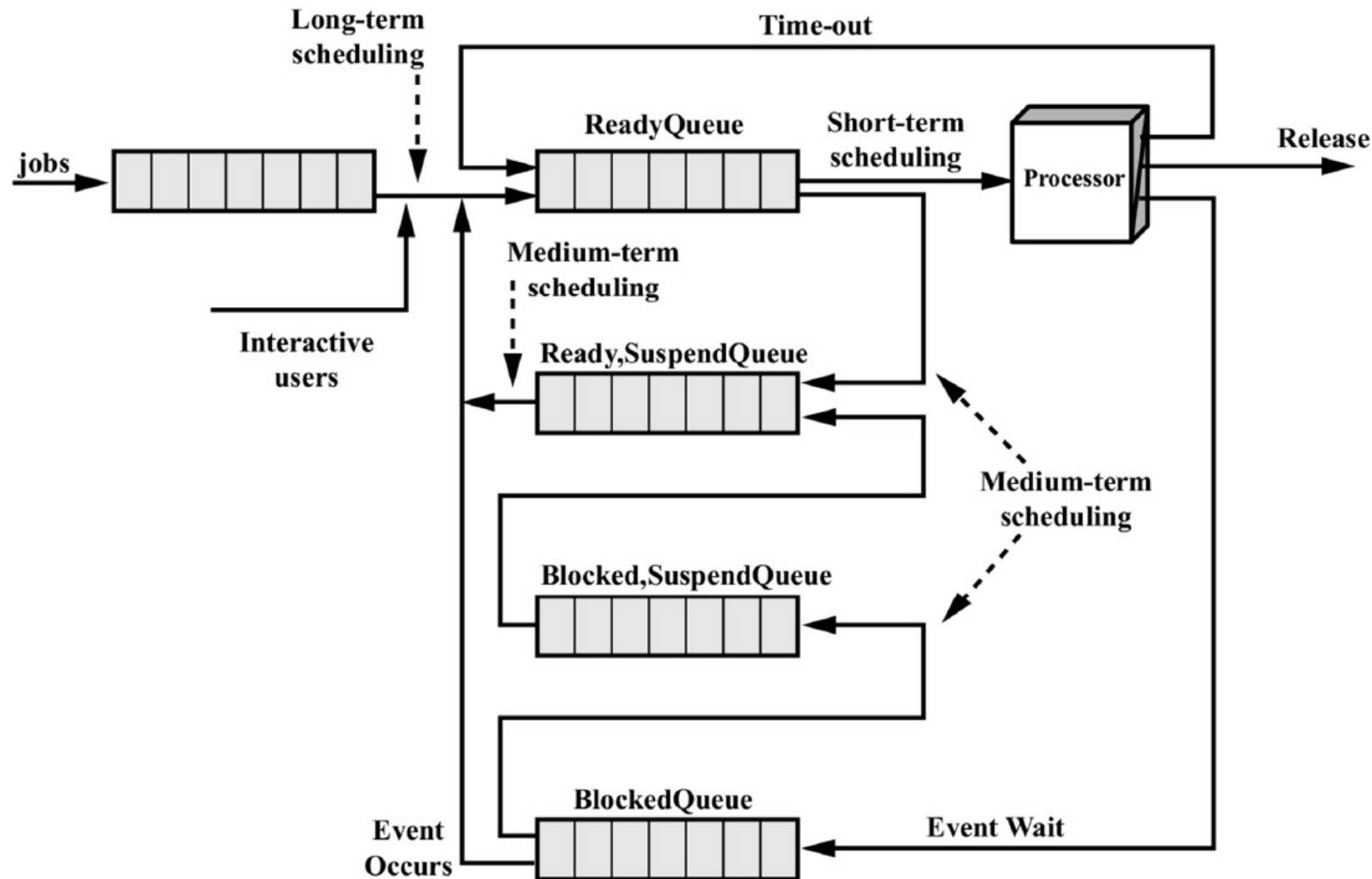
- New → Ready state.
- Decisions:
 - When to activate a process?
 - Which process to activate?
- Monitors idle state of the processor.
- Processor vs. I/O -bound processes



Processor Scheduling

Medium-Term Scheduling

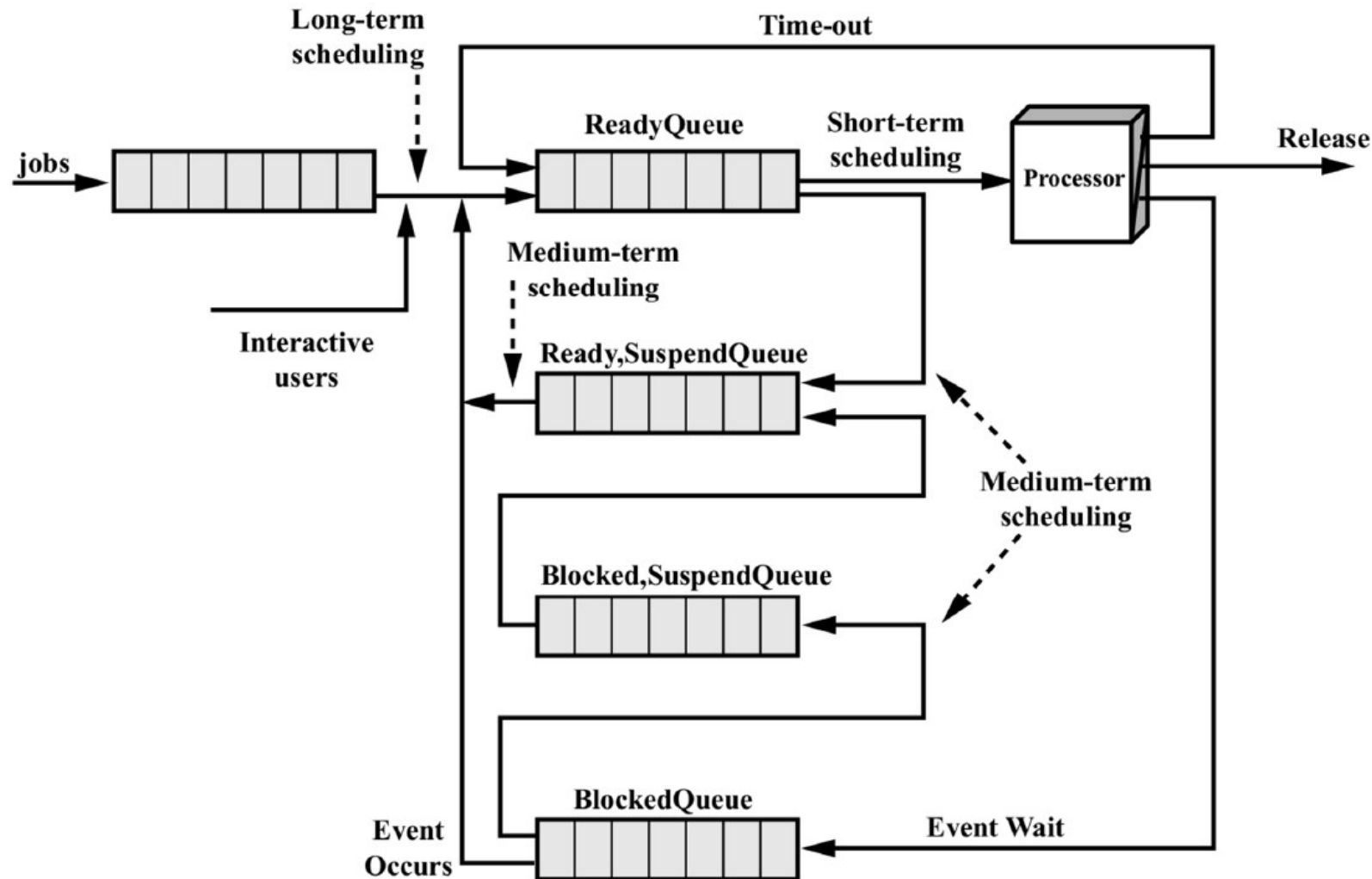
- Occurs more frequently than long-term.
- Processes that move into suspend state
(related to virtual memory management.)



Processor Scheduling

Short-Term Scheduling

- Decides which process can use the processor.
- Various algorithms to do this.



Methodology

Methodology

Performance Metrics

- Average turnaround time T_q .
 - How long it take before a process is completely executed .
 - Includes time a process is blocked or in ready state.
- Response time
 - Time before process goes into running state.
 - Relevant for interactive systems

Methodology

Performance Metrics

- Average turnaround time T_q .
 - How long it take before a process is completely executed .
 - Includes time a process is blocked or in ready state.
- Response time
 - Time before process goes into running state.
 - Relevant for interactive systems

- Fairness

- How resources are distributed among processes.
- Subjective. How to measure? (normalized turnaround time)

- Predictability

- Influence of other processes.
- Ideally: a process has ~constant execution time.

Methodology

Notation: Scheduling Models / Problems

- Universal notation: $\alpha|\beta|\gamma$
 - α : Machine Environment
 - β : Pre-conditions
 - γ : Function to be minimized
- Machine environment (α)
 - $\alpha = 1$, there is just 1 machine/processor
 - $\alpha = Pm$, there are m identical machines/processors

Methodology

Notation: Scheduling Models / Problems

- Pre-conditions (β)
 - $\beta = r_j$, the processes have arrival/*release times*.
 - $\beta = pmtn$, preemption is possible.
 - $\beta = prec$, precedence conditions are applicable. ($p_o \rightarrow p_f$)
 - Deadlines are implicit in function γ
- Function to be minimized (γ)
 - Total completion time ($\sum C_j$)
 - Number of tasks that do not meet a deadline ($\sum U_j$)

Methodology

Notation: Scheduling Models / Problems

- Example: $1 \mid \text{prec}, r_j \mid \sum C_j$
 - A single-processor environment.
 - Not all the tasks arrive at time $t=0$.
 - There are certain tasks that must finish before certain others may start.
(usually indicated via a precedence graph)
 - We want to minimize the total completion time

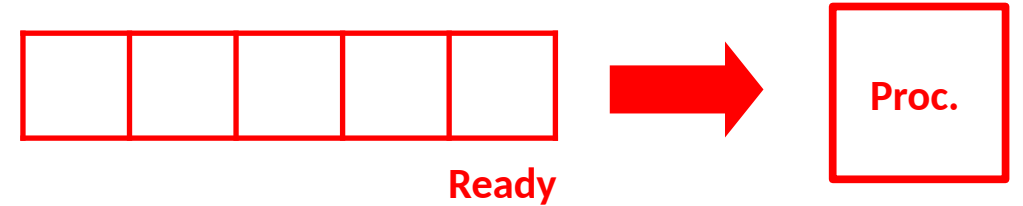
Short-Term Scheduling Algorithms

Non Real-Time Short-Term Scheduling Algorithms

Short-Term Scheduling Algorithms

- **First-In First-Out (FIFO)**

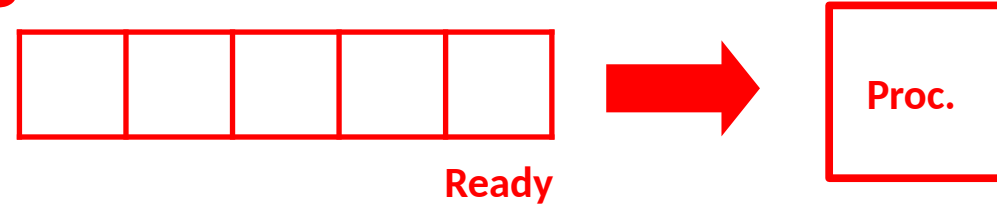
- Oldest process in the ready queue goes first.



- **Properties**

- Small processes may have to wait long. (high turnaround time)
 - I/O bound processes are severely disadvantaged.
(re-queue after every I/O operation)
 - Better when combined with priority mechanisms.

Short-Term Scheduling Algorithms



- **Round Robin (RR)**

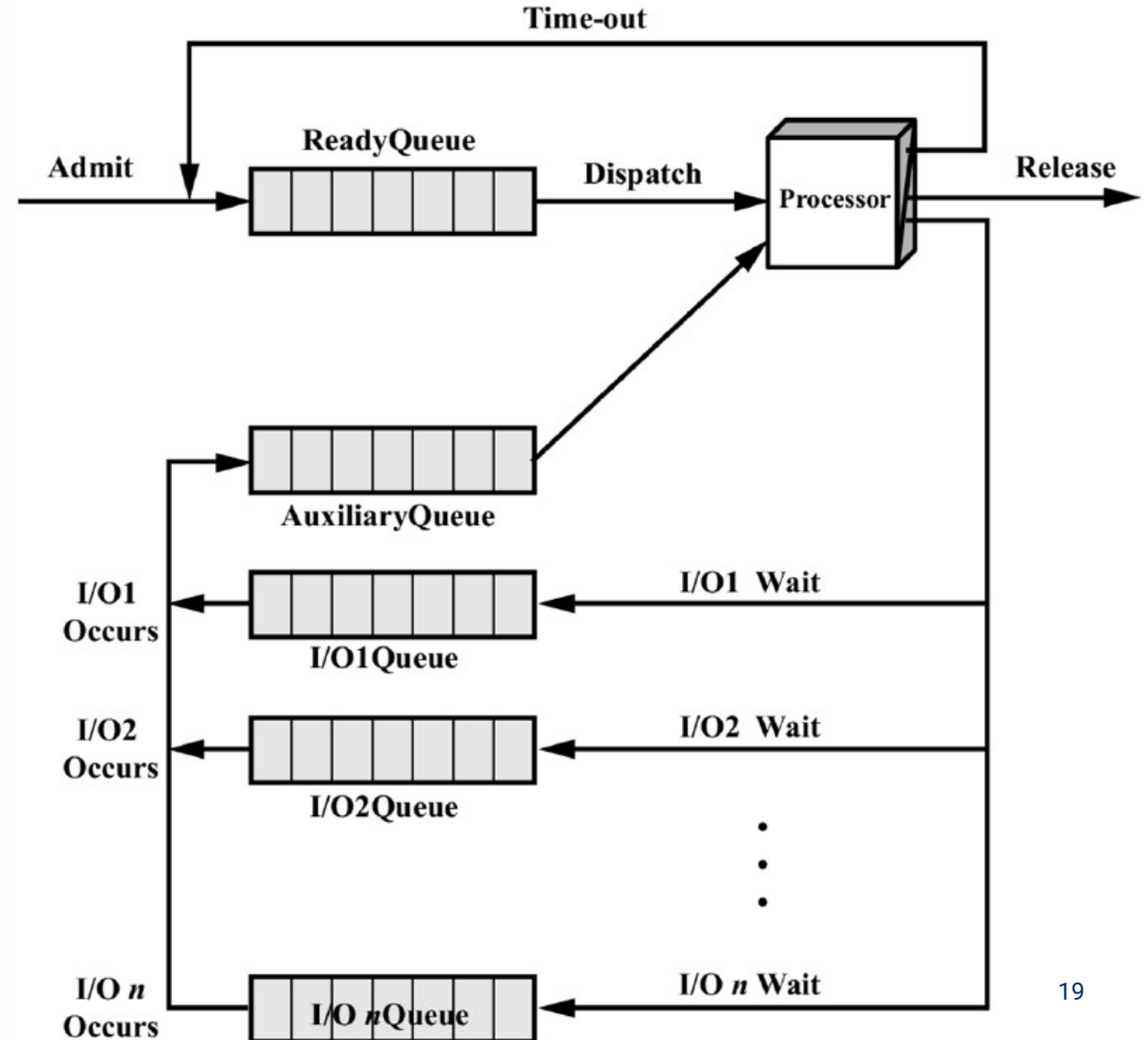
- Each process gets to take turns the processor during a quantum q .
- When the q is up, go to the end of the ready queue.

- **Properties**

- Small processes conclude faster.
- *Choice of q is important ($q \rightarrow 0$: processor sharing)*
- Still disadvantageous for I/O bound processes.
 - I/O operation required before reaching q
 - Addressed by using a separate [priority] queue.

Short-Term Scheduling Algorithms

- Round Robin (RR)
 - *With a separate [priority] queue for I/O bound processes.*



Non Real-Time Short-Term Scheduling Algorithms

(Optimal Algorithms)

Optimal Algorithms

- **Short Process Next (SPN)**

- Choose process with the shortest execution time p first

- **Properties**

- Optimal for $1 || \sum C_j$
 - Batch of tasks arriving together at time $t=0$
 - No preemptive (no interruptions)
- Requires prior knowledge of processing time p of every process.

Optimal Algorithms

- **Short Process Next (SPN) – Optimality**

- Let's consider the Turnaround time $T_q \sim \sum c_j$ (with p_j the processing time of process j)

$$T_q = p_1 + (p_1 + p_2) + \dots + (p_1 + \dots + p_N)$$

Optimal Algorithms

- **Short Process Next (SPN) – Optimality**

- Let's consider the Turnaround time $T_q \sim \sum c_j$ (with p_j the processing time of process j)

$$T_q = p_1 + (p_1 + p_2) + \dots + (p_1 + \dots + p_N) = \sum_{i=1}^N (N - i + 1)p_i$$

Optimal Algorithms

- **Short Process Next (SPN) – Optimality**

- Let's consider the Turnaround time $T_q \sim \sum c_j$ (with p_j the processing time of process j)

$$T_q = p_1 + (p_1 + p_2) + \dots + (p_1 + \dots + p_N) = \sum_{i=1}^N (N - i + 1)p_i$$

- Optimal if $p_1 \leq p_2 \leq \dots \leq p_N$

Optimal Algorithms

- **Short Process Next (SPN) – Optimality**

- Let's consider the Turnaround time $T_q \sim \sum C_j$ (with p_j the processing time of process j)

$$T_q = p_1 + (p_1 + p_2) + \dots + (p_1 + \dots + p_N) = \sum_{i=1}^N (N - i + 1)p_i$$

- Optimal if $p_1 \leq p_2 \leq \dots \leq p_N$
- If $r_j > 0 \rightarrow$ not optimal ($1 | r_j | \sum C_j$)
 - Example: assume $p_1=100$, $p_2=1$, $r_1=0$ and $r_2=1$

Optimal Algorithms

- **Short Process Next (SPN) – Optimality**

- Let's consider the Turnaround time $T_q \sim \sum C_j$ (with p_j the processing time of process j)

$$T_q = p_1 + (p_1 + p_2) + \dots + (p_1 + \dots + p_N) = \sum_{i=1}^N (N - i + 1)p_i$$

- Optimal if $p_1 \leq p_2 \leq \dots \leq p_N$
- If $r_j > 0 \rightarrow$ not optimal ($1 | r_j | \sum C_j$)
 - Example: assume $p_1=100$, $p_2=1$, $r_1=0$ and $r_2=1$
 - if execute directly $[p_1, p_2] \rightarrow \sim T_q = (100+100)/2 = 100$.

Optimal Algorithms

- **Short Process Next (SPN) – Optimality**

- Let's consider the Turnaround time $T_q \sim \sum C_j$ (with p_j the processing time of process j)

$$T_q = p_1 + (p_1 + p_2) + \dots + (p_1 + \dots + p_N) = \sum_{i=1}^N (N - i + 1)p_i$$

- Optimal if $p_1 \leq p_2 \leq \dots \leq p_N$

- If $r_j > 0 \rightarrow$ not optimal ($1 | r_j | \sum C_j$)

- Example: assume $p_1=100$, $p_2=1$, $r_1=0$ and $r_2=1$
 - if execute directly $[p_1, p_2] \rightarrow \sim T_q = (100+100) = 200$
 - If wait until $t=1$ $[p_2, p_1] \rightarrow \sim T_q = (1+102) = 103$

Optimal Algorithms

- **Weighted Short Process Next (WSPN)**

- Similar to SPN, but now each process has a weight w_j
- Aims at minimizing the weighted turnaround time $T_q \sim 1/N \sum w_j c_j$

$$w_1 p_1 + w_2(p_1 + p_2) + \dots + w_N(p_1 + \dots + p_N)$$

- **Properties**

- Optimal for $1 \parallel \sum w_j c_j$
- Smith's Rule (1956): Batch scheduling optimal according to p_j/w_j

Optimal Algorithms

- **Weighted Short Process Next (WSPN)**

- Similar to SPN, but now each process has a weight w_j
- Aims at minimizing the weighted turnaround time $T_q \sim 1/N \sum w_j c_j$

$$w_1 p_1 + w_2(p_1 + p_2) + \dots + w_N(p_1 + \dots + p_N)$$

$$w_i(p_1 + p_2 + \dots + p_{i-1} + p_i)$$

- **Properties**

- Optimal for $1 \parallel \sum w_j c_j$
- Smith's Rule (1956): Batch scheduling optimal according to p_j/w_j

Optimal Algorithms

- **Weighted Short Process Next (WSPN) – Optimality**

Consider the ordering **S** that is optimal but not WSPN, then

$$\frac{p_i}{w_i} > \frac{p_{i+1}}{w_{i+1}}$$

Optimal Algorithms

- **Weighted Short Process Next (WSPN) – Optimality**

Consider the ordering **S** that is optimal but not WSPN, then

$$\frac{p_i}{w_i} > \frac{p_{i+1}}{w_{i+1}} \rightarrow p_i w_{i+1} > p_{i+1} w_i$$

Optimal Algorithms

- **Weighted Short Process Next (WSPN) – Optimality**

Consider the ordering **S** that is optimal but not WSPN, then

$$\frac{p_i}{w_i} > \frac{p_{i+1}}{w_{i+1}} \rightarrow p_i w_{i+1} > p_{i+1} w_i$$

turnaround time for i and $i+1$

$$w_i(p_1 + p_2 + \dots + p_{i-1} + p_i) + w_{i+1}(p_1 + p_2 + \dots + p_{i-1} + p_i + p_{i+1})$$

Optimal Algorithms

- **Weighted Short Process Next (WSPN) – Optimality**

Consider the ordering **S** that is optimal but not WSPN, then

$$\frac{p_i}{w_i} > \frac{p_{i+1}}{w_{i+1}} \rightarrow p_i w_{i+1} > p_{i+1} w_i$$

turnaround time for i and $i+1$

$$w_i(p_1 + p_2 + \dots + p_{i-1} + p_i) + w_{i+1}(p_1 + p_2 + \dots + p_{i-1} + p_i + p_{i+1})$$

Consider the ordering **S'** obtained from switching processes i and $i+1$ in **S**.

$$w_{i+1}(p_1 + p_2 + \dots + p_{i-1} + p_{i+1}) + w_i(p_1 + p_2 + \dots + p_{i-1} + p_{i+1} + p_i)$$

Optimal Algorithms

- **Weighted Short Process Next (WSPN) – Optimality**

Consider the ordering **S** that is optimal but not WSPN, then

$$\frac{p_i}{w_i} > \frac{p_{i+1}}{w_{i+1}} \rightarrow p_i w_{i+1} > p_{i+1} w_i$$

turnaround time for i and $i+1$

$$w_i(p_1 + p_2 + \dots + p_{i-1} + p_i) + w_{i+1}(p_1 + p_2 + \dots + p_{i-1} + p_i + p_{i+1})$$

Consider the ordering **S'** obtained from switching processes i and $i+1$ in **S**.

$$w_i(p_1 + p_2 + \dots + p_{i-1} + p_{i+1} + p_i) + w_{i+1}(p_1 + p_2 + \dots + p_{i-1} + p_{i+1})$$

Optimal Algorithms

- **Weighted Short Process Next (WSPN) – Optimality**

Consider the ordering **S** that is optimal but not WSPN, then

$$\frac{p_i}{w_i} > \frac{p_{i+1}}{w_{i+1}} \rightarrow p_i w_{i+1} > p_{i+1} w_i$$

turnaround time for i and $i+1$

$$w_i(p_1 + p_2 + \dots + p_{i-1} + p_i) + w_{i+1}(p_1 + p_2 + \dots + p_{i-1} + p_i + p_{i+1})$$

Consider the ordering **S'** obtained from switching processes i and $i+1$ in **S**.

$$w_i(p_1 + p_2 + \dots + p_{i-1} + p_{i+1} + p_i) + w_{i+1}(p_1 + p_2 + \dots + p_{i-1} + p_{i+1})$$

- **Subtracting S – S'**

$$w_{i+1}p_i - w_i p_{i+1}$$

Optimal Algorithms

- **Weighted Short Process Next (WSPN) – Optimality**

Consider the ordering **S** that is optimal but not WSPN, then

$$\frac{p_i}{w_i} > \frac{p_{i+1}}{w_{i+1}} \rightarrow p_i w_{i+1} > p_{i+1} w_i$$

turnaround time for i and $i+1$

$$w_i(p_1 + p_2 + \dots + p_{i-1} + p_i) + w_{i+1}(p_1 + p_2 + \dots + p_{i-1} + p_i + p_{i+1})$$

Consider the ordering **S'** obtained from switching processes i and $i+1$ in **S**.

$$w_i(p_1 + p_2 + \dots + p_{i-1} + p_{i+1} + p_i) + w_{i+1}(p_1 + p_2 + \dots + p_{i-1} + p_{i+1})$$

- **Subtracting S – S'**

$$w_{i+1}p_i - w_i p_{i+1} > 0 \rightarrow T_q \text{ decreased}$$

Optimal Algorithms

- **Weighted Short Process Next (WSPN) – Optimality**

Consider the ordering **S** that is optimal but not WSPN, then

$$\frac{p_i}{w_i} > \frac{p_{i+1}}{w_{i+1}} \rightarrow p_i w_{i+1} > p_{i+1} w_i$$

turnaround time for i and $i+1$

$$w_i(p_1 + p_2 + \dots + p_{i-1} + p_i) + w_{i+1}(p_1 + p_2 + \dots + p_{i-1} + p_i + p_{i+1})$$

Consider the ordering **S'** obtained from switching processes i and $i+1$ in **S**.

$$w_i(p_1 + p_2 + \dots + p_{i-1} + p_{i+1} + p_i) + w_{i+1}(p_1 + p_2 + \dots + p_{i-1} + p_{i+1})$$

- **Subtracting S – S'**

$$w_{i+1}p_i - w_i p_{i+1} > 0 \rightarrow T_q \text{ decreased} \Rightarrow S \text{ was not optimal}$$

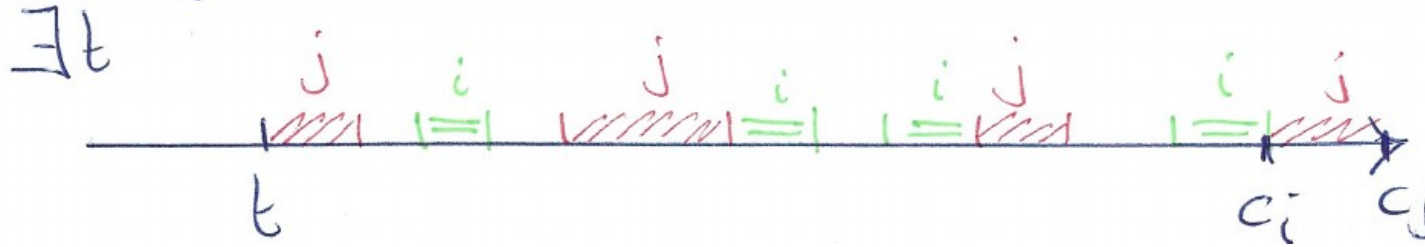
Optimal Algorithms

- **Shortest Remaining Time Next (SRTN)**
 - Select the process that has smallest remaining execution time.
 - Preemptive
- **Properties**
 - Optimal for $1 |r_j, pmtn| \sum c_j$
 - Requires knowledge of length of tasks
 - Remaining processing time must be tracked
 - Starvation occurs for longer tasks.

Optimal Algorithms

- Shortest Remaining Time Next (SRTN) – Optimality

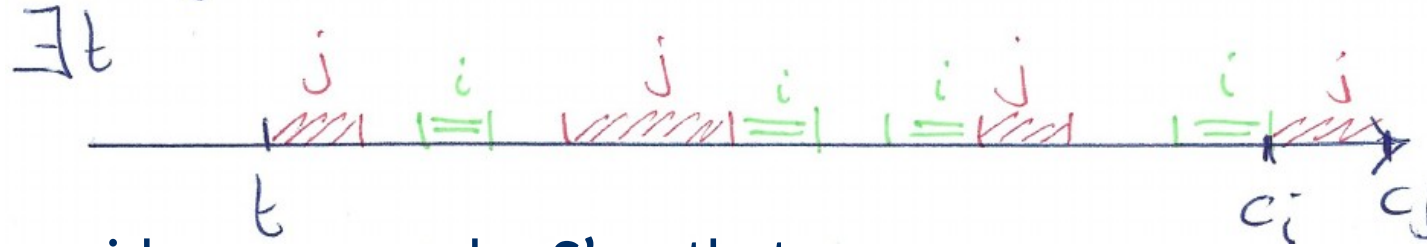
Consider the ordering S that is optimal but not SRTN, then



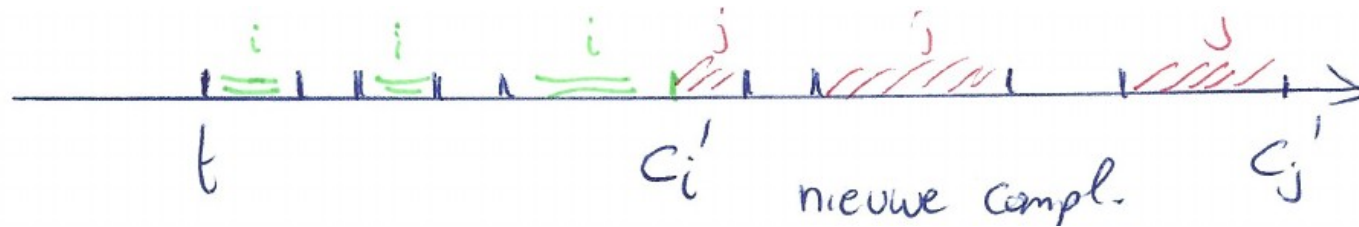
Optimal Algorithms

- Shortest Remaining Time Next (SRTN) – Optimality

Consider the ordering S that is optimal but not SRTN, then



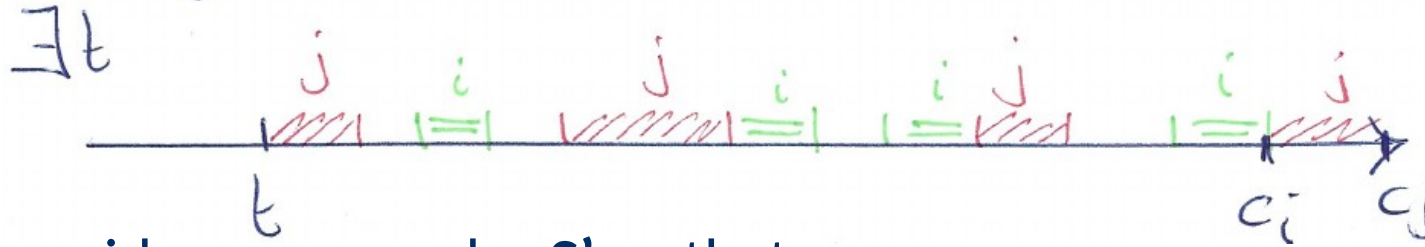
consider a new order S' so that



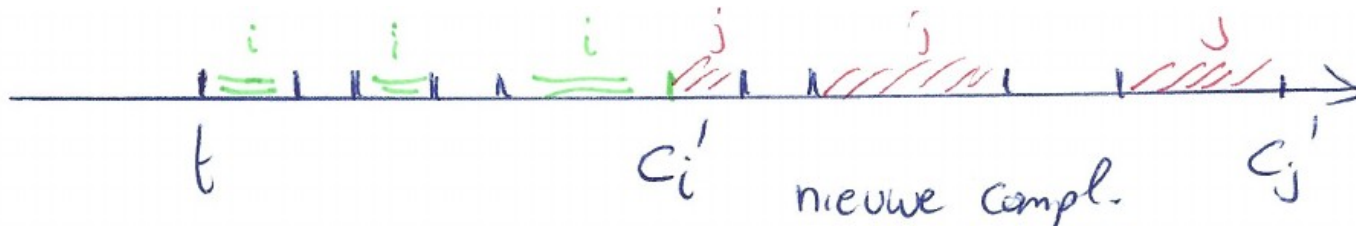
Optimal Algorithms

- Shortest Remaining Time Next (SRTN) – Optimality

Consider the ordering S that is optimal but not SRTN, then



consider a new order S' so that

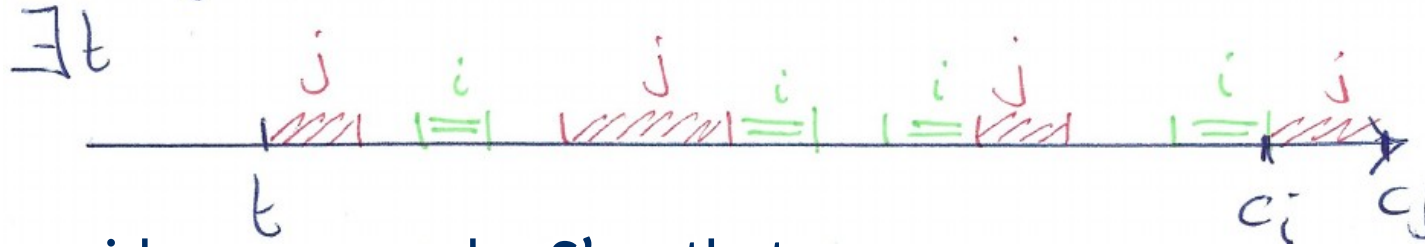


$$C_j' = \max(C_i, C_j)$$

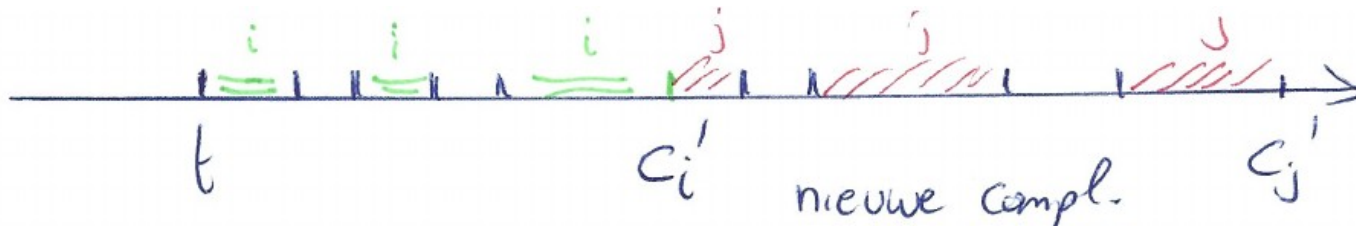
Optimal Algorithms

- Shortest Remaining Time Next (SRTN) – Optimality

Consider the ordering S that is optimal but not SRTN, then



consider a new order S' so that



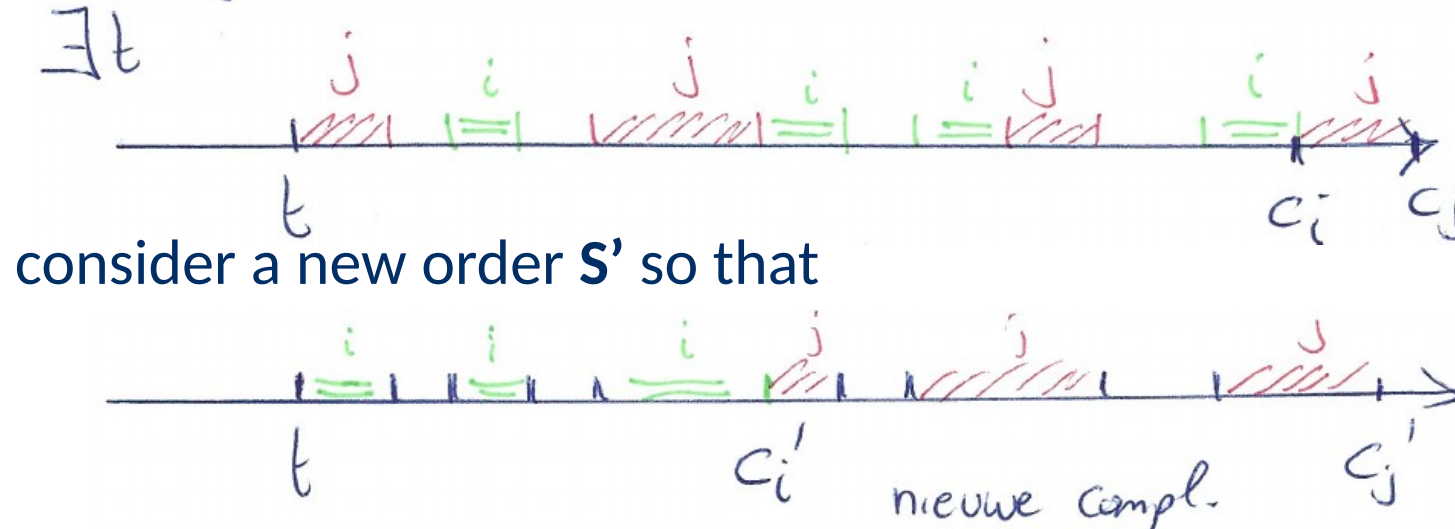
$$C_j' = \max(C_i, C_j)$$

$$C_i' < \min(C_i, C_j)$$

Optimal Algorithms

- Shortest Remaining Time Next (SRTN) – Optimality

Consider the ordering S that is optimal but not SRTN, then



$$C_j' = \max(C_i, C_j)$$

$$C_i' < \min(C_i, C_j)$$

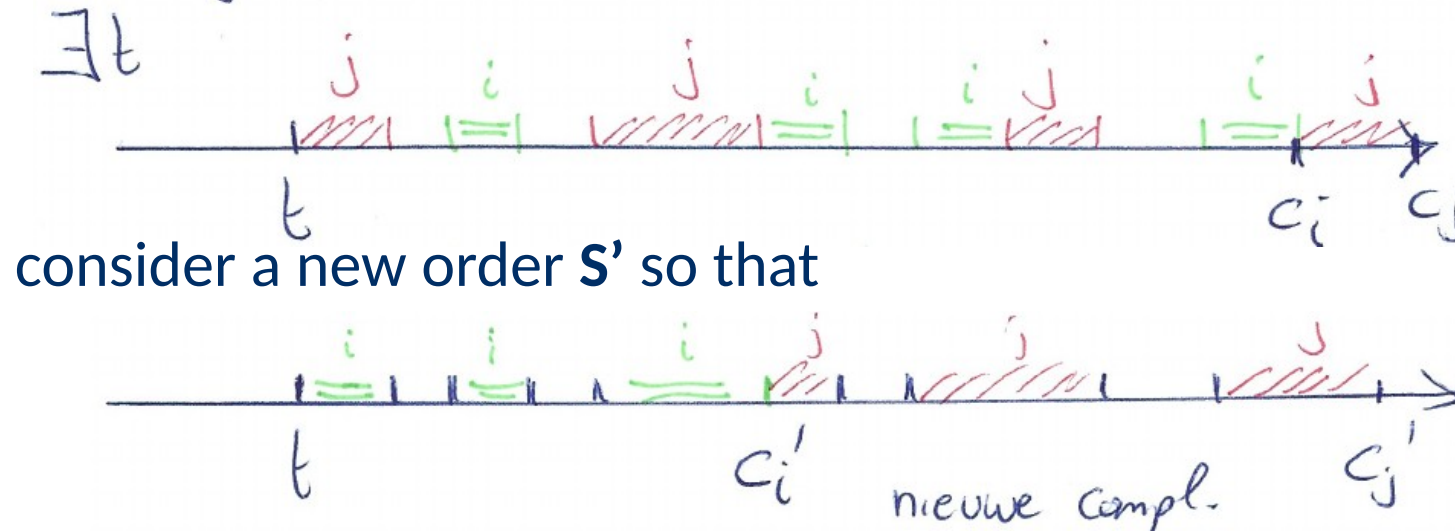
- Comparing the turnaround times

$$\sum_k C_k = \sum_{k \neq i, j} C_k + C_i + C_j > \sum_{k \neq i, j} C_k + C_i' + C_j'.$$

Optimal Algorithms

- Shortest Remaining Time Next (SRTN) – Optimality

Consider the ordering S that is optimal but not SRTN, then



$$C'_j = \max(C_i, C_j)$$

$$C'_i < \min(C_i, C_j)$$

- Comparing the turnaround times

$$\sum_k C_k = \sum_{k \neq i, j} C_k + C_i + C_j > \sum_{k \neq i, j} C_k + C'_i + C'_j.$$

$\rightarrow T_q$ decreased
 $\Rightarrow S$ was not optimal

Optimal Algorithms

- **Non-preemptive SRTN**

- Minimize running time when tasks are not interrupted
- Given the optimal order with completion times: $C_1 \leq C_2 \leq \dots \leq C_N$
- Construct the same sequence with completion times $\bar{C}_1 \leq \bar{C}_2 \leq \dots \leq \bar{C}_N$
possibly leaving the processor idle while task in question arrives

- **Properties**

- Optimal rule is NP-hard
- Possible to achieve an order with completion time at most as twice as large as the optimal (*2-approximation*)

$$\sum_j \bar{C}_j \leq 2 \sum_j C_j$$

Optimal Algorithms

- **Non-preemptive SRTN – Optimality**

- We will prove that $\sum_j \bar{C}_j \leq 2 \sum_j C_j$

Optimal Algorithms

▪ Non-preemptive SRTN – Optimality

- We will prove that $\sum_j \bar{C}_j \leq 2 \sum_j C_j$
- Considering the completion time C_j

$$\bar{C}_j \leq \max_{k \leq j} r_k + \sum_{k=1}^j p_k$$

Optimal Algorithms

▪ Non-preemptive SRTN – Optimality

- We will prove that $\sum_j \bar{C}_j \leq 2 \sum_j C_j$
- Considering the completion time C_j

$$\bar{C}_j \leq \max_{k \leq j} r_k + \sum_{k=1}^j p_k$$

Maximal if the first task is the last of the j-task to arrive

Optimal Algorithms

▪ Non-preemptive SRTN – Optimality

- We will prove that $\sum_j \bar{C}_j \leq 2 \sum_j C_j$
- Considering the completion time C_j

$$\bar{C}_j \leq \max_{k \leq j} r_k + \sum_{k=1}^j p_k$$

Maximal if the first task is the last of the j-task to arrive

- For $k \leq j \rightarrow r_k \leq C_k \leq C_j \rightarrow \max_{k \leq j} r_k \leq C_j$

Optimal Algorithms

▪ Non-preemptive SRTN – Optimality

- We will prove that $\sum_j \bar{C}_j \leq 2 \sum_j C_j$
- Considering the completion time C_j

$$\bar{C}_j \leq \max_{k \leq j} r_k + \sum_{k=1}^j p_k$$

Maximal if the first task is the last of the j-task to arrive

- For $k \leq j \rightarrow r_k \leq C_k \leq C_j \rightarrow \max_{k \leq j} r_k \leq C_j$
- Since the tasks are processed in order $\rightarrow \sum_{k=1}^j p_k \leq C_j$

Optimal Algorithms

▪ Non-preemptive SRTN – Optimality

- We will prove that $\sum_j \bar{C}_j \leq 2 \sum_j C_j$
- Considering the completion time C_j

$$\bar{C}_j \leq \max_{k \leq j} r_k + \sum_{k=1}^j p_k$$

Maximal if the first task is the last of the j-task to arrive

- For $k \leq j \rightarrow r_k \leq C_k \leq C_j \rightarrow \max_{k \leq j} r_k \leq C_j$
 - Since the tasks are processed in order $\rightarrow \sum_{k=1}^j p_k \leq C_j$
- $\rightarrow \bar{C}_j \leq 2C_j$

Optimal Algorithms

- **Highest Response Ratio Next (HRRN)**

- Uses the weighted turnaround time $RR_s = (w + p_s) / p_s$
- W increases with waiting time, proportional to p_s .
- Choose process with larger RR .

- **Properties**

- Requires knowledge of p (or an estimate)
- Good for small processes

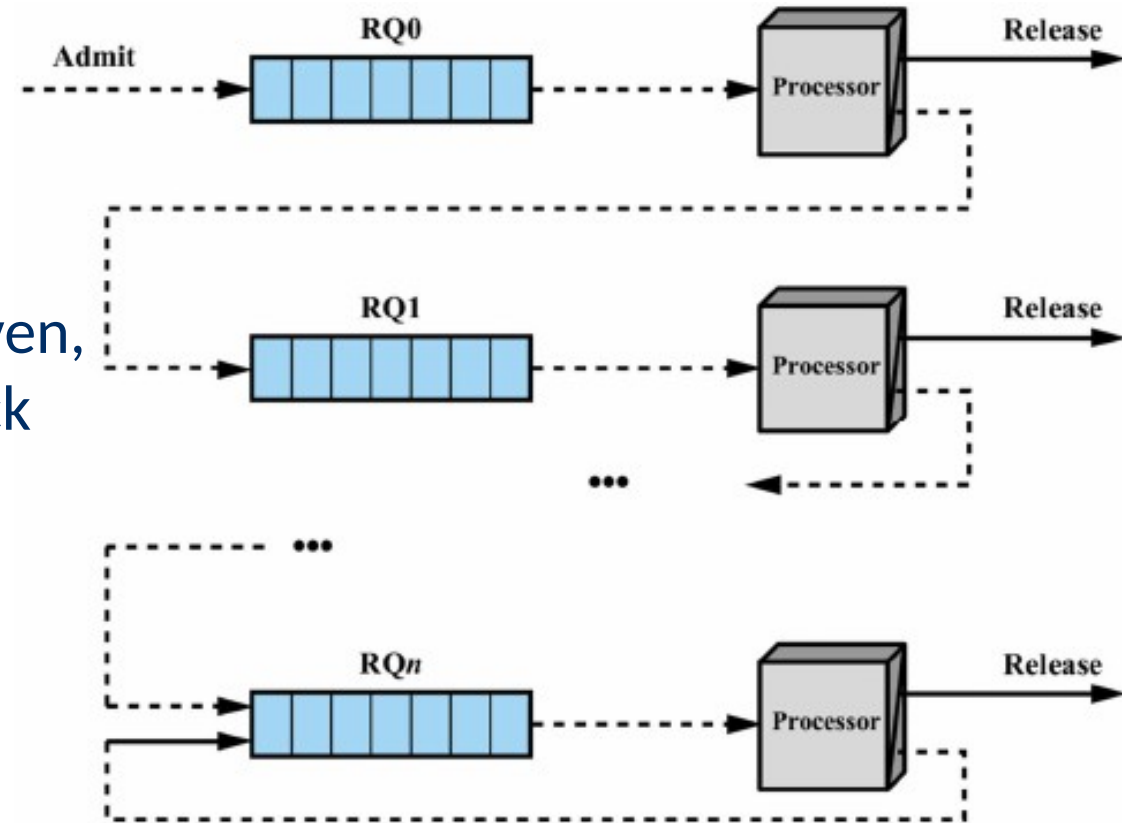
Optimal Algorithms

▪ Multilevel Feedback Queues

- Mimic behavior of previous schedules **without having execution times p_j and arrival times r_j .**
- The more time a process was already given, the less quickly we let it get back on track

▪ Implementation

- N priority FIFO queues.
- Access to the processor for quantum q , then go to queue $i+1$.
- Last queue is round robin.



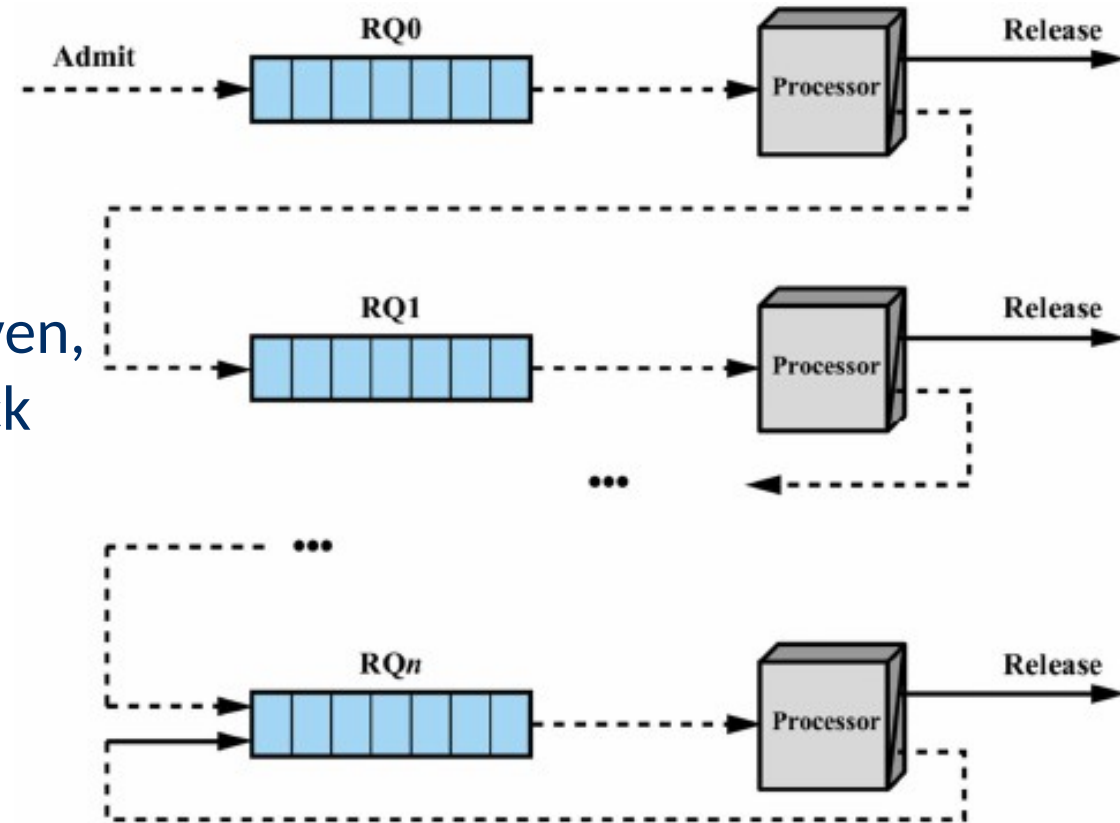
Optimal Algorithms

■ Multilevel Feedback Queues

- Mimic behavior of previous schedules **without having execution times p_j and arrival times r_j .**
- The more time a process was already given, the less quickly we let it get back on track

■ Implementation

- N priority FIFO queues.
- Access to the processor for quantum q , then go to queue $i+1$.
- Last queue is round robin.

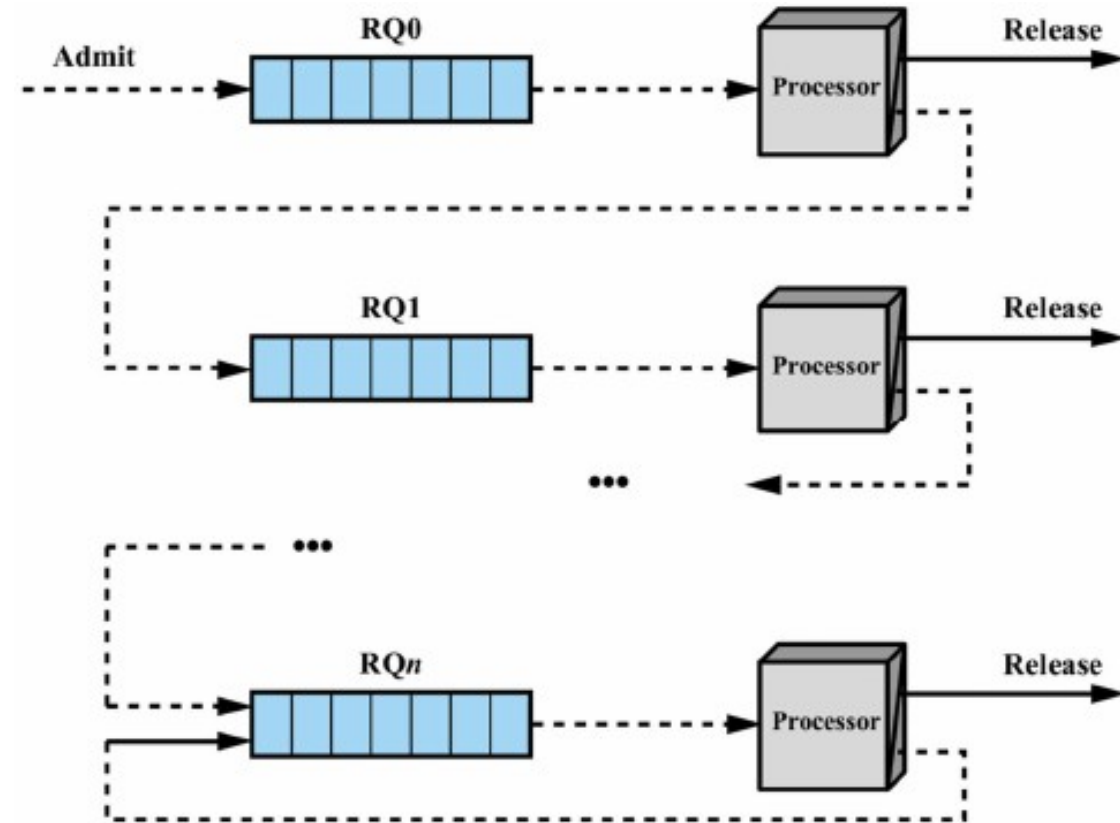


How would long/short processes perform in this setting?

Optimal Algorithms

▪ Multilevel Feedback Queues

- Increasing quantum q , helps with the starvation of long processes.
- I/O processes go back to queue 1 after the I/O operation.



Traditional Unix Scheduler

- Related to multilevel feedback queue, use priorities
- Divide time into fixed intervals (e.g., 1 sec)
 - Within an interval i , each process j has a fixed priority $P_j(i)$, (low value = high priority)
 - A counter $U_j(i)$, is also maintained
- Within each interval fixed number of clock interrupts (e.g. every 1/60 sec.)
- Counter $U_j(i)$ of the **active process** j , is increased by one with each interruption.
- At each clock interruption the process with the highest priority will use the processor.

Traditional Unix Scheduler

- At the end of each interval i , we adjust the priority:

$$P_j(i) = Base_j + CPU_j(i-1)/2 + nice_j$$

with

$$CPU_j(i) = U_j(i)/2 + CPU_j(i-1)/2 = (U_j(i) + CPU_j(i-1))/2.$$

- $Base_j$: divide processes into classes (kernel, swap, I/O support, etc.)
- $Nice_j$: configurable by the user
- Starvation of large processes is prevented via $CPU_j(i)$.

Traditional Unix Scheduler

Fairness for Groups of Processes

- Goal: distribute (proportionally) the CPU capacity to groups of processes (group = single user or research group, organization)
- Add extra counter $GU_k(i)$ per group k
 - Increase if process from group k gets interrupted
 - Adjust $GCPU_k(i)$ after each interval:

$$GCPU_k(i) = (GU_k(i) + GCPU_k(i-1)) / 2$$

- Priority of process then becomes (W_k = weight)

$$P_j(i) = Base_j + GCPU_k(i) / 4W_k + CPU_j(i-1) / 2 + nice_j$$

Key points

[Take Home Message]

Processor Scheduling

Pay attention to...

- Advantages and disadvantages of FIFO, Round Robin, HRRF and multilevel feedback queues (and variants from them).
- UNIX scheduler: operation, fairness, relationship to the above methods.
- SPN, WSPN, SRTN and nSRTN: optimality contexts and proofs

Further Reading

Essential: Scheduling

- OS Course Notes - chapter 5.
- Blackboard – Recorded lecture from Prof. Van Houdt.

Exercise Sessions: Tomorrow (04/11)

- Complementary Session on Paging
- Session on Scheduling planned for 18/11

Questions?



Operating Systems

[1500WETOPS]

José Oramas